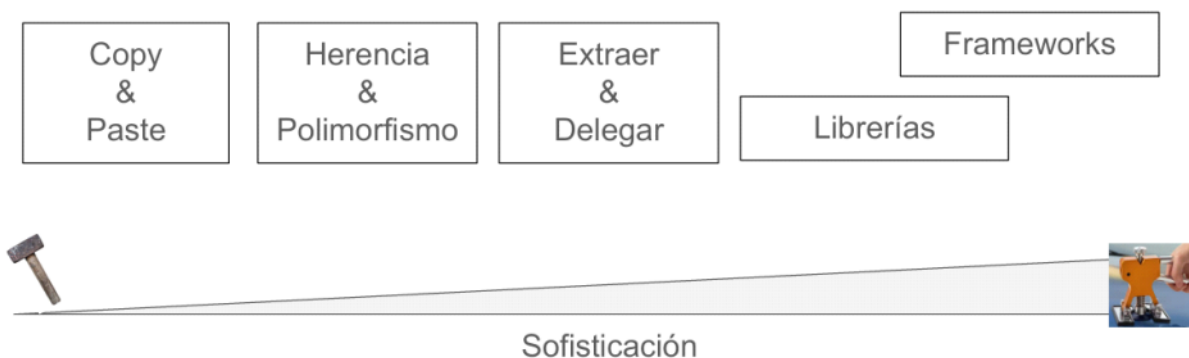


El reuso es inesperado

Modos de reuso



Framework: Es una aplicación "semi-completa", "reusable", que puede ser especializada para producir aplicaciones a medida...

Un conjunto de clases concretas y abstractas, relacionadas para proveer una arquitectura reusable que implementa una familia de aplicaciones (relacionadas).

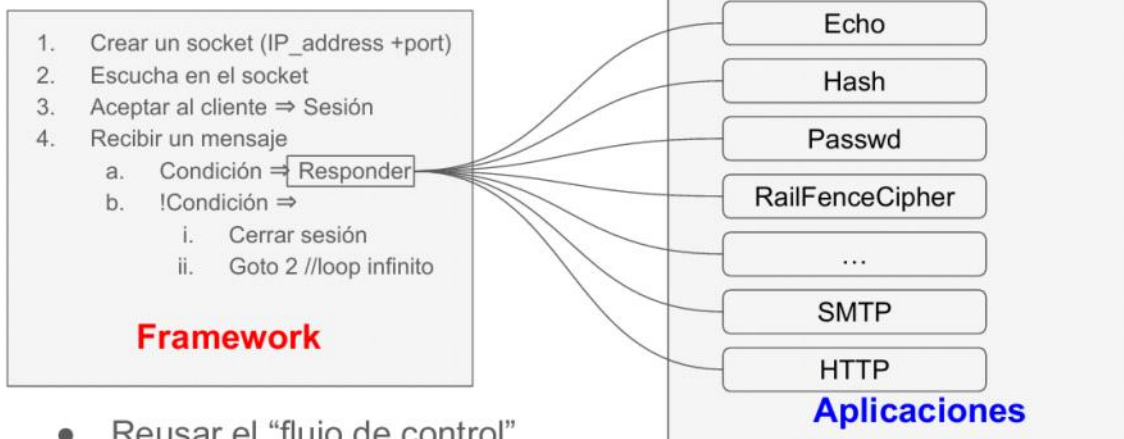
Foco del diseño

- Una aplicación está diseñada para responder a requerimientos de usuarios
- Una librería está diseñada para proveer reuso de cierto tipo de funcionalidad (código fuente). Nuestro código "llama" o "invoca" funcionalidad de la librería
 - Autenticación
 - Cliente LLM
 - Conectividad a Base de Datos
- Un framework está diseñado para proveer reuso de una manera de ejecutar (arquitectura). Se provee código o se configura para que ejecute
 - Application Frameworks (tienen **loop** de control)
 - Integration / Infrastructure Frameworks
 - En todos los casos tiene un método (hilo) que implementa "inversion de control" (Hollywood Principle)

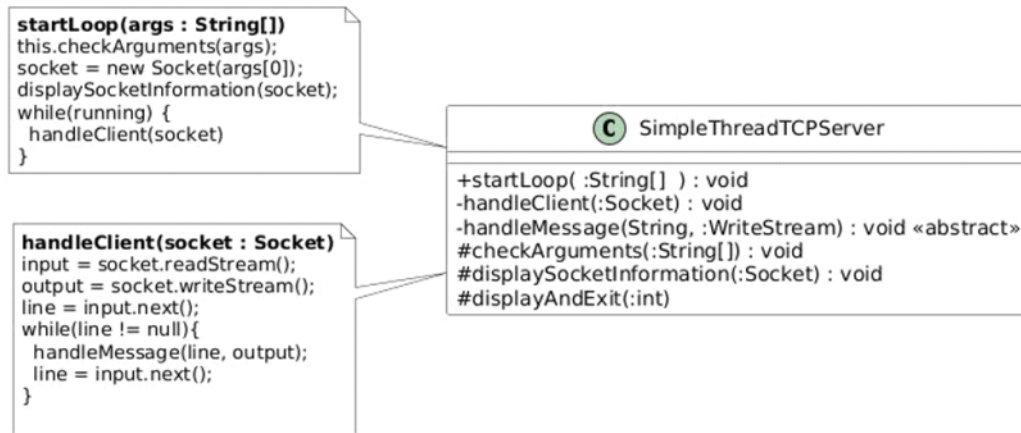
Aplicación: Servidores TCP

- Es un programa que implementa una de las partes de la arquitectura Cliente/Servidor
 - Un Servidor escucha en un socket TCP (IP_address+port)
 - Un Cliente establece una conexión con el servidor para enviar mensajes (plain text)
 - El Servidor responde
- Ejemplos:
 - SMTPServer: emails
 - DayTimeServer: fecha actual
 - HTTPServer: envío y recepción de documentos web
 - Backend applications
 - Servicios / Cloud microservices
 - EchoServer (helloworld de este tipo de aplicaciones):
 - Recibe un mensaje de texto
 - Responde con el mismo mensaje que el cliente envió

Flujo de Control (típico)



- Reusar el “flujo de control”
 - El flujo de control es incompleto (generalmente no compila)
 - “Responder” es donde se puede “enganchar” la funcionalidad
 - “Responder” es un hook (obligatorio)
- El flujo de control pertenece a un framework



- handleMessage() es donde se implementa la funcionalidad
 - Antes del concepto de framework, se modificaba el código :- (
 - Con OOP + Frameworks, se pueden crear subclases
 - Cada subclase es una “instanciacion” del framework

SimpleThreadTCPServer (framework)

Flujo de Control (incompleto)

1. Crear un socket (IP_address +port)
2. Escucha en el socket
3. Aceptar al cliente ⇒ Sesión
4. Recibir un mensaje
 - a. Condición ⇒ **Responder**
 - b. !Condición ⇒ Cerrar sesión

Instanciación (Cookbook)

1. Subclasificar SimpleThreadTCPServer
 - a. Debe implementar Main(String[])
 - i. crear una instancia
 - ii. enviar método startLoop(String[])
 - b. Debe implementar handleMessage(String)
 - c. Métodos que podría implementar
 - i. acceptAndDisplaySocket(ServerSocket)
 - ii. checkArguments(String[])
 - iii. displayAndExit(int)
 - iv. displaySocketData(Socket)
 - v. displayUsage()
 - vi. displaySocketInformation()
 - vii. displayWarning()

SimpleThreadTCPServer

```
12 public final void startLoop(String[] args) {
13     checkArguments(args);
14
15     int portNumber = Integer.parseInt(args[0]);
16
17
18     try (ServerSocket serverSocket = new ServerSocket(portNumber)) {
19         displaySocketInformation(portNumber);
20         while (true) {
21             Socket clientSocket = acceptAndDisplaySocket(serverSocket);
22             handleClient(clientSocket);
23         }
24     } catch (IOException e) {
25         displayAndExit(portNumber);
26     }
27 }
```

1. Todos los servidores pueden redefinir:
 - a. checkArguments(), displaySocketInformation() y displayAndExit()

SimpleThreadTCPServer.handleClient(Socket)

```
62 private final void handleClient(Socket clientSocket) {
63
64     try {
65         PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
66         BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream())); {
67             String inputLine;
68             while ((inputLine = in.readLine()) != null) {
69                 System.out.println("Received message: " + inputLine + " from "
70                     + clientSocket.getInetAddress().getHostAddress() + ":" + clientSocket.getPort());
71                 handleMessage(inputLine, out);
72                 if (inputLine.equalsIgnoreCase("")) {
73                     break; // Client requested to close the connection
74                 }
75             }
76             System.out.println("Connection closed with " + clientSocket.getInetAddress().getHostAddress() + ":"
77                 + clientSocket.getPort());
78         } catch (IOException e) {
79             System.err.println("Problem with communication with client: " + e.getMessage());
80         } finally {
81             try {
82                 clientSocket.close();
83             } catch (IOException e) {
84                 System.err.println("Error closing socket: " + e.getMessage());
85             }
86         }
87     }
88 }
89 }
```

1. Todos los servidores debern definir:
 - a. handleMessage(String, PrintWriter)

EchoServer.java

```
1 import java.io.PrintWriter;
2
3 public class EchoServer extends SingleThreadTCPServer {
4
5     public void handleMessage(String message, PrintWriter out) {
6         out.println(message);
7     }
8
9     public static void main(String[] args) {
10         new EchoServer().startLoop(args);
11     }
12 }
13
14
```

1. startLoop(args) arranca el loop de control del framework
 - a. El Framework toma el control (**inversión de control**)
 - b. Ahora está completo porque se implementa handleMessage(...)
2. handleMessage(...) es invocado desde alguna parte del framework
 - a. Completa el loop de control
 - b. “Hollywood Principle”: no nos llames, nosotros te contactaremos
 - c. EchoServer hereda el loop de control no existe encapsulamiento ⇒ es una **Caja Blanca**
 - d. La ejecución del server depende de la correcta implementación de handleMessage() porque es parte del loop (incompleto) de control

Framework de Caja Blanca:

- La instanciación hereda y completa el "hilo" de control
 - El hilo de control invoca al código
- Es posible que requiera agregar métodos a clases del framework
- Demanda conocimiento del código del framework

Es una Caja Blanca.

HotSpots vs FrozenSpot

FrozenSpot: aspecto del framework que afecta a todas las instanciaciones y que no se puede modificar (marca indeleble)

Ej: SingleThreadTCPServer cierra la conexión con el cliente si:

1. El cliente cierra la conexión
2. El cliente envía un mensaje vacío ¹

—Todas las aplicaciones de ester framework tienen esta “marca”/arquitectura

```
72 if (inputLine.equalsIgnoreCase("")) {
73     break; // Client requested to close the connection
74 }
```

Ejercicio/Evaluacion: como podría parametrizar la “condición de cierre de sesión” ?

(1) SingleThreadTCPServer.java: 72-74

HotSpots vs FrozenSpot

HotSpot: estructura en el código que permite modificar el comportamiento del framework, para instanciar o para extender.

Ej: SingleThreadTCPServer podría¹ usar una jerarquía de “políticas” para determinar cuando una sesión debe cerrarse.

- Toda instanciación deberían elegir la política de cierre de session
- Nuevas políticas de cierre de sesión se pueden agregar para “extender” el framework



(1) SingleThreadTCPServer.java: no tiene implementado esta modificación en la versión publicada.

Resumiendo SimpleThreadTCPServer

- El método startLoop()
 - Definición única de como ejecutan todas las instanciaciones
 - Invoca al método handleClient() que invoca handleMessage()
 - De manera indirecta implementa “inversión de control” (es el hilo de control)
- HandleMessage()
 - Es un “gancho” para todas las instanciaciones
 - Todas las instanciaciones deben implementar ese método
- Existen otros métodos que pueden ser reimplementados por las instanciaciones de los que existen implementaciones por defecto.
 - acceptAndDisplaySocket(ServerSocket)
 - checkArguments(String[])
 - displayAndExit(int)
 - displaySocketData(Socket)
 - displayUsage()
 - displaySocketInformation()
 - displayWarning()

Resumen de Frameworks

- Proveen una solución reusable para una familia de aplicaciones
- Las clases en el framework se relacionan (herencia, conocimiento, envío de mensajes) de manera que resuelven la mayor parte del problema en cuestión
- El código del framework controla/usa al código de la instanciación
- Instanciacion(Framework) = Aplicación/Solución
- Extension(Framework) = Framework'
- Tipos de Frameworks
 - Aplicación: desktop, webapps, tcpservers :-)
 - Manejo Datos: ORDB, pipelines, NRDB
 - Sistemas Distribuidos: mensajes, eventos, rpc
 - Testing: unit, web pages
- oo2.next()
 - Blackbox frameworks: instanciación por composición
 - Frameworks & Design Patterns



Diseños de Frameworks:

- **FrozenSpots:** Partes del diseño que NO cambian.
- **HotSpots:** Elementos del diseño pensados para adaptarse
 - Hook Methods
 - Patrones de Diseño (Clases/Jerarquías, delegación, protocolos)

Reuso: Framework II (Caja Negra):

Resumen de la clase anterior

Frameworks:

- Proveen una solución reusable para una familia de aplicaciones
- Las clases en el framework se relacionan (herencia, conocimiento, envío de mensajes) de manera que resuelven la mayor parte del problema en cuestión
- El código del framework controla/usa al código de la instanciación
- Tipos de Frameworks
 - Aplicación: desktop, webapps, tcpservers :-)
 - Manejo Datos: ORDB, pipelines, NRDB
 - Sistemas Distribuidos: mensajes, eventos, rpc
 - Testing: unit, web pages
- Framework de Caja Blanca: las instanciaciones “completan” el hilo de control agregando código.
 - Ejercitando un hotspot con herencia
 - Modificando código fuente del framework

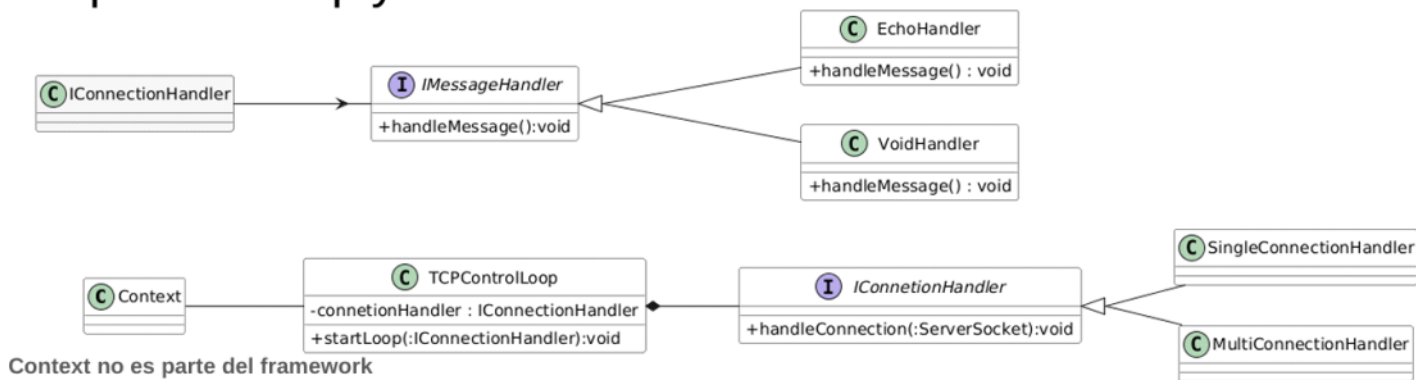
```
1 import java.io.PrintWriter;
2
3 public class EchoServer extends SingleThreadTCPServer {
4
5     public void handleMessage(String message, PrintWriter out) {
6         out.println(message);
7     }
8
9     public static void main(String[] args) {
10
11         new EchoServer().startLoop(args);
12
13     }
14 }
```

SingleThreadTCPServer
(hotspot herencia)

```
1 import tcp.server.reply.*;
2
3 public class EchoApp {
4
5     public static void main(String[] args) {
6
7         new TCPControlLoop(new SingleConnectionHandler(new EchoHandler())).startLoop(args);
8
9     }
10 }
```

tcp.server.reply
(hotspot composición)

tcp.server.reply



FrozenSpot:

Condición de corte de la conexión
 Un tipo de conexión (runtime)
 Un tipo de MessageHandler (runtime)

HotSpot:

Nuevos MessageHandlers s/funcionalidad
 Hash, timestamp, etc
 Nuevos ConnectionHandlers
 Timeout, recording, etc

COMPLETAR

Resumiendo I

- Frameworks presentan una manera de reuso que supera el reuso (solo de código)
- Un Framework dicta la manera de ejecutar un programa (execution thread)
- El diseño de un framework foco en una manera de describir un dominio o en las cosas importantes de un dominio
- Frameworks vistos en OO2
 - SingleThreadTCPServer: Un servidor como extensión de una clase loop + sesión (singular)
 - Tcp.server.reply: loop(sesión(msgHandler)). Sesión singular o multiple. Diferentes MsgHandler
 - Java.util.logging: jerarquía de Labels + composición de filtros, formatos y salidas
- Diseños de Frameworks
 - FrozenSpots: partes del diseño que no cambian
 - HotSpots: elementos del diseño pensadas para adaptarse
 - Hook Methods
 - Patrones de Diseño (Clases/Jerarquías, Delegación, Protocolos)
 - El diseño se va adaptando a los problemas comunes en un dominio
 - Problemas comunes ⇒ design patterns
 - En CajaBlanca. Loop de control suele ser Template Method.
 - En tcp.server.reply MessageHandler puede ser Strategy o Command (GoF)
 - Si en el diseño de un framework identificamos un patrón, la estructura correspondiente seguramente es un hotspot.

5)5)b)

El método **register** NO es un método hook. Porque no hay ningún motivo para redefinir ese **register**

Cuando extendiendo el Framework desaparecen los conceptos de hotspot, frozen spot, caja negra y caja blanca.

TCP Server es de **caja blanca**.

Si es abstracto seguramente sea un hook.

Extensión por Composición y Configuración **caja negra**

Hablamos de un enfoque de **caja negra** cuando el Framework se extiende mediante la composición y delegación de objetos. Configuramos el framework componiendo objetos que el mismo framework provee. No tenemos que saber nada acerca de cómo están implementados.

Extensión por Herencia **caja blanca**

Extender puede ser implementar los hook methods o subclasificar la herencia.

Instanciar un framework es instanciar.

Cuando extendiendo el Framework estoy asumiendo el rol de "Desarrollador" y el resultado es una nueva versión del Framework.